

NUR Hand-in 1

Berend Nieuwhof

February 28, 2025

Abstract

In this document, the solutions to the first hand-in exercise are presented.

1 Exercise 1: Poisson distribution

In this section, the solutions to the first exercise are presented.

Our script is given by:

```
1 '''This file implements a function that returns the poisson distribution for an integer
2 k.'''
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.stats import poisson
6
7 def log_factorial(k):
8     '''Computes the factorial of a number in log space.'''
9     if k == 0:
10         result = 0
11     else:
12         k_values = np.arange(k, 0, -1)
13         result = np.sum(np.log(k_values))
14     return np.float32(result)
15
16 # using log space makes us less likely to have overflow errors.
17
18 def log_poisson(lamb, k):
19     '''Computes the log of the poisson distribution for a positive mean lambda.'''
20     assert lamb > 0
21     k = np.int32(k)
22
23     result = k * np.log(lamb) - lamb - log_factorial(k)
24     return np.float32(result)
25
26 def poisson(lamb, k):
27     return np.float32(np.exp(log_poisson(lamb, k)))
28
29 lamb_k_values = np.array([np.array([1, 5, 3, 2.6, 100, 101], dtype=np.float32),
30                           np.array([0, 10, 21, 40, 5, 200], dtype=np.int32)])
31
32 np.set_printoptions(precision=8)
33
34 for i in range(len(lamb_k_values[0])):
35     lamb = lamb_k_values[0, i]
36     k = lamb_k_values[1, i]
37     poisson_val = poisson(lamb, k)
38     print(f'for lamb = {lamb:2f} and k = {k:1f}, P.lambda(k) = {poisson_val}'.format())
```

Poisson.py

As apparent from the code above, we have adopted a computation in log space. This avoids possible overflow errors and enables us to compute the Poisson probability for larger (λ, k) pairs. The output for a representative sample of pairs is given below, which demonstrates that the returned Poisson probability is accurate when compared to calculations done in online calculators.

```

1 for lamb = 1.000000 and k = 0.000000, P_lambda(k) = 0.3678794205188751
2 for lamb = 5.000000 and k = 10.000000, P_lambda(k) = 0.018132776021957397
3 for lamb = 3.000000 and k = 21.000000, P_lambda(k) = 1.019340254565515e-11
4 for lamb = 2.600000 and k = 40.000000, P_lambda(k) = 3.61510320423337e-33
5 for lamb = 100.000000 and k = 5.000000, P_lambda(k) = 3.1000581991562844e-36
6 for lamb = 101.000000 and k = 200.000000, P_lambda(k) = 1.2695239237170379e-18

```

poisson_output.txt

2 Exercise 2: Vandermonde Matrix

In this section we look at exercise 2, which is about Vandermonde matrices. The full script is presented below, and the parts of it specific to subquestions will be discussed in appropriate subsections.

Our script is given by:

```

1 # Main code body and plotting code adapted from Marcel van Daalen, at
2 # https://home.strw.leidenuniv.nl/~daalen/Handin_files/vandermonde.py
3
4 import numpy as np
5 import sys
6 import os
7 import matplotlib.pyplot as plt
8 import copy
9 import timeit
10
11 def crout(A):
12     m, n = A.shape
13     for i in range(m):
14         for j in range(n):
15             if i >= j: # if i > j, add the alpha[i, j] element
16                 A[i, j] = A[i, j] - np.sum(A[i, :j]*A[:j, j])
17             else: # else, edit matrix beta
18                 A[i, j] = (A[i, j] - np.sum(A[i, :i]*A[:i, j]))/A[i, i]
19
20     return A
21
22 def forward_sub2(A, b):
23     m, _ = A.shape
24     y = copy.deepcopy(b)
25     for k in range(m):
26         y[k] = (b[k] - np.sum(A[k, :k]*y[:k]))/A[k, k]
27     return y
28
29 def backward_sub2(A, b):
30     _, n = A.shape
31     x = np.zeros_like(b)
32     for i in range(n-1, -1, -1):
33         x[i] = b[i] - np.dot(A[i, i+1:], x[i+1:])
34     return x
35
36 def get_coeffs(y, A):
37     A_decomp = crout(A)
38     fw_sub = forward_sub2(A_decomp, y)
39     coeffs = backward_sub2(A_decomp, fw_sub)
40
41     return coeffs, A_decomp
42
43 def polynomial(x, coeffs):
44     y = np.zeros_like(x)
45     for i in range(len(x)):
46         for j in range(len(coeffs)):
47             y[i] += coeffs[j] * x[i]**j
48     return y
49
50 # Let's get started
51 data=np.genfromtxt(os.path.join(sys.path[0], "Vandermonde.txt"), comments='#', dtype=np.
    float64)

```

```

52 |
53 | x=data[:,0]
54 | y=data[:,1]
55 |
56 | # Construct the Vandermonde matrix.
57 | A = np.zeros([20, 20])
58 | m, n = A.shape
59 | for i in range(m):
60 |     for j in range(n):
61 |         A[i, j] = x[i]**j
62 |
63 | A_orig = copy.deepcopy(A)
64 |
65 | # now set up the polynomial function!
66 | coeffs, crout_LU = get_coeffs(y, A)
67 | xx=np.linspace(x[0],x[-1],1001) #x values to interpolate at
68 | yya = polynomial(xx, coeffs)
69 | ya= polynomial(x, coeffs)
70 |
71 | # Now we'd like to implement neville's algorithm.
72 |
73 | def neville(datax, datay, x_interp):
74 |     y_interp = []
75 |     error_est = []
76 |     for x in x_interp:
77 |         y = datay.copy()
78 |         M = len(datax)
79 |
80 |         for k in range(1, M):
81 |             for i in range(M-k):
82 |                 y[i] = ((x - datax[i+k])*y[i] + (datax[i] - x)*y[i+1]) / (datax[i] -
83 | datax[i+k])
84 |             y_interp.append(y[0])
85 |             error_est.append(np.abs(y[0] - y[1])) # also save an error estimate given by the
86 | last improvement
87 |         return y_interp, error_est
88 |
89 | yyb, yyb_err= neville(x, y, xx)
90 | yb, yb_err= neville(x, y, x)
91 |
92 | # Let's implement the error canceling algorithm.
93 |
94 | def error_cancel(A_orig, crout_LU, y, coeffs, iterations):
95 |     for _ in range(iterations): # We solve the system  $Ac - y = v$  for  $v$ , to find
96 | corrections on the coefficients
97 |         v = A_orig @ coeffs - y
98 |         fw_result = forward_sub2(crout_LU, v)
99 |         coeff_corr = backward_sub2(crout_LU, fw_result)
100 |         coeffs = coeffs - coeff_corr
101 |     return coeffs
102 |
103 | c1_coeffs = error_cancel(A_orig, crout_LU, y, coeffs, iterations=1)
104 | yyc1= polynomial(xx, c1_coeffs)
105 | yc1= polynomial(x, c1_coeffs)
106 |
107 | c10_coeffs = error_cancel(A_orig, crout_LU, y, coeffs, iterations=10)
108 | yyc10= polynomial(xx, c10_coeffs)
109 | yc10= polynomial(x, c10_coeffs)
110 |
111 | print('Coefficients found with LU method (a):\n', coeffs)
112 |
113 | print('Coefficients found with 1 iteration of LU correction (c):\n', c1_coeffs)
114 |
115 | print('Coefficients found with 10 iterations of LU correction (c):\n', c10_coeffs)
116 | #Plot of points with absolute difference shown on a log scale (question 2a)
117 | fig=plt.figure()
118 | gs=fig.add_gridspec(2, hspace=0, height_ratios=[2.0, 1.0])
119 | axs=gs.subplots(sharex=True, sharey=False)
120 | axs[0].plot(x,y,marker='o',linewidth=0)

```

```

119 plt.xlim(-1,101)
120 axs[0].set_ylim(-400,400)
121 axs[0].set_ylabel('$y$')
122 axs[1].set_ylim(1e-16,1e1)
123 axs[1].set_ylabel('$|y-y_i|$')
124 axs[1].set_xlabel('$x$')
125 axs[1].set_yscale('log')
126 line,=axs[0].plot(xx,yya,color='orange')
127 line.set_label('Via LU decomposition')
128 axs[0].legend(frameon=False,loc="lower left")
129 axs[1].plot(x,abs(y-ya),color='orange')
130 plt.savefig('my_vandermonde_sol.2a.png',dpi=600)
131
132 #For questions 2b and 2c, add this block
133 line,=axs[0].plot(xx,yyb,linestyle='dashed',color='green')
134 line.set_label('Via Neville\'s algorithm')
135 axs[0].legend(frameon=False,loc="lower left")
136 axs[1].plot(x,abs(y-yb),linestyle='dashed',color='green')
137 plt.savefig('my_vandermonde_sol.2b.png',dpi=600)
138
139 #For question 2c, add this block too
140 line,=axs[0].plot(xx,yc1,linestyle='dotted',color='red')
141 line.set_label('LU with 1 iteration')
142 axs[1].plot(x,abs(y-yc1),linestyle='dotted',color='red')
143 line,=axs[0].plot(xx,yc10,linestyle='dashdot',color='purple')
144 line.set_label('LU with 10 iterations')
145 axs[1].plot(x,abs(y-yc10),linestyle='dashdot',color='purple')
146 axs[0].legend(frameon=False,loc="lower left")
147 plt.savefig('my_vandermonde_sol.2c.png',dpi=600)
148
149 #Don't forget to caption your figures to describe them/
150 #mention what conclusions you draw from them!
151
152 # Now, finally, time the different methods.
153
154 def problem_2a():
155     coeffs, crout_LU = get_coeffs(y, A)
156     yya = polynomial(xx, coeffs)
157
158 def problem_2b():
159     yyb, yyb_err= neville(x, y, xx)
160
161 def problem_2c():
162     coeffs, crout_LU = get_coeffs(y, A)
163     c10_coeffs = error_cancel(A_orig, crout_LU, y, coeffs, iterations=10)
164     yyc10= polynomial(xx, c10_coeffs)
165
166 number = 150
167 two_a_time = timeit.timeit(lambda : problem_2a(), number = number)
168
169 two_b_time = timeit.timeit(lambda : problem_2b(), number = number)
170
171 two_c_time = timeit.timeit(lambda : problem_2c(), number = number)
172
173 print(f'Time for {number} iterations of 2a:', two_a_time)
174 print(f'Time for {number} iterations of 2b:', two_b_time)
175 print(f'Time for {number} iterations of 2c:', two_c_time)

```

Vandermonde.py

2.1 2a: fitting by LU decomposition

The relevant code is given by:

```

1 import numpy as np
2 import sys
3 import os
4 import matplotlib.pyplot as plt
5 import copy

```

```

6 import timeit
7
8 def crout(A):
9     m, n = A.shape
10    for i in range(m):
11        for j in range(n):
12            if i >= j: # if i > j, add the alpha[i, j] element
13                A[i, j] = A[i, j] - np.sum(A[i, :j]*A[:j, j])
14            else: # else, edit matrix beta
15                A[i, j] = (A[i, j] - np.sum(A[i, :i]*A[:i, j]))/A[i, i]
16
17    return A
18
19 def forward_sub2(A, b):
20     m, _ = A.shape
21     y = copy.deepcopy(b)
22     for k in range(m):
23         y[k] = (b[k] - np.sum(A[k, :k]*y[:k]))/A[k, k]
24     return y
25
26 def backward_sub2(A, b):
27     _, n = A.shape
28     x = np.zeros_like(b)
29     for i in range(n-1, -1, -1):
30         x[i] = b[i] - np.dot(A[i, i+1:], x[i+1:])
31     return x
32
33 def get_coeffs(y, A):
34     A_decomp = crout(A)
35     fw_sub = forward_sub2(A_decomp, y)
36     coeffs = backward_sub2(A_decomp, fw_sub)
37
38     return coeffs, A_decomp
39
40 def polynomial(x, coeffs):
41     y = np.zeros_like(x)
42     for i in range(len(x)):
43         for j in range(len(coeffs)):
44             y[i] += coeffs[j] * x[i]**j
45     return y
46
47 # Let's get started
48 data=np.genfromtxt(os.path.join(sys.path[0], "Vandermonde.txt"), comments='#', dtype=np.
49                    float64)
50
51 x=data[:,0]
52 y=data[:,1]
53
54 # Construct the Vandermonde matrix.
55 A = np.zeros([20, 20])
56 m, n = A.shape
57 for i in range(m):
58     for j in range(n):
59         A[i, j] = x[i]**j
60
61 A_orig = copy.deepcopy(A)
62
63 # now set up the polynomial function!
64 coeffs, crout_LU = get_coeffs(y, A)
65 xx=np.linspace(x[0], x[-1], 1001) #x values to interpolate at
66 yya = polynomial(xx, coeffs)

```

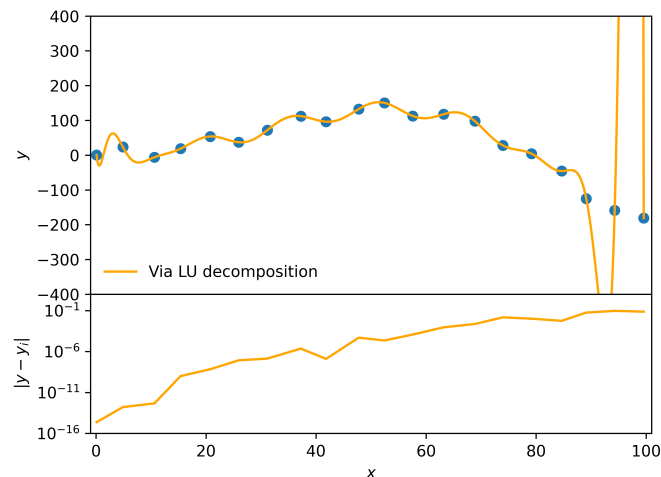
Vandermonde.py

We start by reading in the data points and constructing the $N \times N$ Vandermonde matrix with each entry of the form $V_{ij} = x_i^j$, with i the row index and j the column index (both up to $N - 1$). We then find the coefficients for the unique polynomial that goes through all given points by solving the system $\mathbf{Vc} = \mathbf{y}$ with $\mathbf{y}$ the given y -values for the points.

To do this, we overwrite the Vandermonde matrix by its LU decomposition using Crout's algorithm. We then solve the system by applying forward substitution, followed by backward substitution.

The result is the unique polynomial given in figure ??.

Figure 1: Upper half: the given points in blue, with the polynomial interpolation using LU decomposition in orange. Bottom half: the absolute difference between the interpolation and the given y value at the given points.



```

1 Coefficients found with LU method (a):
2 [ 1.70147517e+01 -1.86599980e+02  2.37038854e+02 -1.09410937e+02
3   2.67337487e+01 -4.05711019e+00  4.16236505e-01 -3.03795406e-02
4   1.63051551e-03 -6.57955383e-05  2.02524984e-06 -4.79493741e-08
5   8.75580154e-10 -1.22942095e-11  1.31443054e-13 -1.05004562e-15
6   6.06574150e-18 -2.39219555e-20  5.76088051e-23 -6.38865757e-26]
```

Vandermonde_output.txt

2.2 2b: fitting by Neville's algorithm

The relevant code is given by:

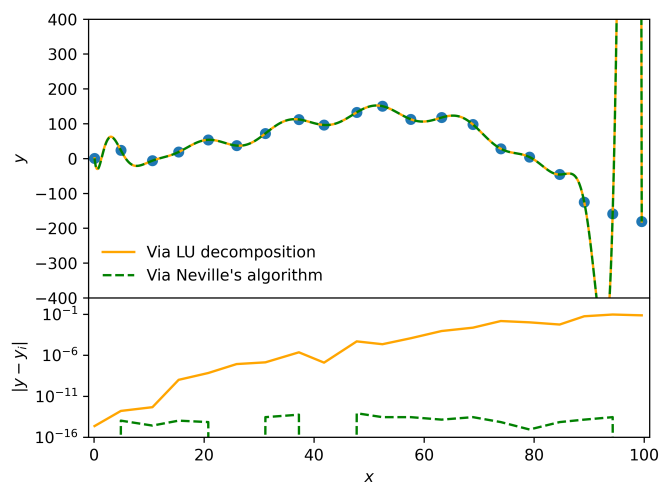
```

1 # Now we'd like to implement neville's algorithm.
2
3
4 def neville(datax, datay, x_interp):
5     y_interp = []
6     error_est = []
7     for x in x_interp:
8         y = datay.copy()
9         M = len(datax)
10
11         for k in range(1, M):
12             for i in range(M-k):
13                 y[i] = ((x - datax[i+k])*y[i] + (datax[i] - x)*y[i+1]) / (datax[i] -
14                 datax[i+k])
15                 y_interp.append(y[0])
16                 error_est.append(np.abs(y[0] - y[1])) # also save an error estimate given by the
17                 last improvement
18         return y_interp, error_est
19
20 yyb, yyb_err= neville(x, y, xx)
21 yb, yb_err= neville(x, y, x)
```

Vandermonde.py

Here, we simply pass an array of values to be interpolated into our Neville interpolation function. The resulting polynomial lies on top of the polynomial found in 2a. This is to be expected, as there is a single unique 19th order polynomial that goes through all of these points. See figure ?? below:

Figure 2: Upper half: the given points in blue, with the polynomial interpolation using LU decomposition in orange. The polynomial found by Neville's algorithm has been added, and we see that it lies on top of the previous one. Bottom half: the absolute difference between the interpolation and the given y value at the given points. The absolute difference for the second method is orders of magnitude smaller.



```

1 Coefficients found with 1 iteration of LU correction (c):
2 [ 1.73261060e+01 -1.89903192e+02  2.39192506e+02 -1.10031275e+02
3   2.68385894e+01 -4.06885525e+00  4.17169348e-01 -3.04342928e-02
4   1.63295507e-03 -6.58795060e-05  2.02750625e-06 -4.79969536e-08
5   8.76367347e-10 -1.23043727e-11  1.31544235e-13 -1.05080648e-15
6   6.06991727e-18 -2.39377260e-20  5.76454210e-23 -6.39259513e-26]
```

Vandermonde_output.txt

Possible reasons for the difference in error are discussed further on in this document.

2.3 2c: error-cancellation using LU decomposition

The relevant code is given by:

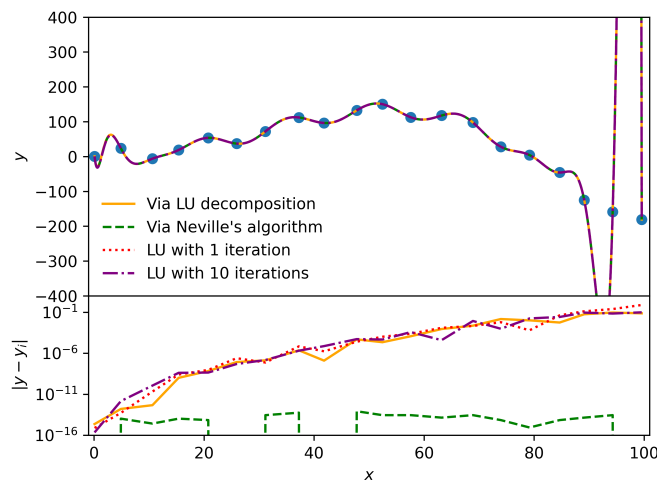
```

1 # Let's implement the error canceling algorithm.
2
3
4 def error_cancel(A_orig, crout_LU, y, coeffs, iterations):
5     for _ in range(iterations): # We solve the system Ac - y = v for v, to find
6         # corrections on the coefficients
7         v = A_orig @ coeffs - y
8         fw_result = forward_sub2(crout_LU, v)
9         coeff_corr = backward_sub2(crout_LU, fw_result)
10        coeffs = coeffs - coeff_corr
11    return coeffs
12
13 c1_coeffs = error_cancel(A_orig, crout_LU, y, coeffs, iterations=1)
14 yyc1= polynomial(xx, c1_coeffs)
15 yc1= polynomial(x, c1_coeffs)
16
17 c10_coeffs = error_cancel(A_orig, crout_LU, y, coeffs, iterations=10)
18 yyc10= polynomial(xx, c10_coeffs)
19 yc10= polynomial(x, c10_coeffs)
```

Vandermonde.py

We now iterate on the solution of 2a. We can use the LU matrices calculated previously and solve a new system to find a correction on the coefficients. We do this once with 1 iteration, and then once more with 10 iterations of the error-cancelling algorithm. The results are shown in figure ??.

Figure 3: Upper half: the given points in blue, with the polynomial interpolation using LU decomposition in orange, as well as the polynomial found by Neville's algorithm. We have now added 2 additional interpolations, for different iterations of the LU error canceling algorithm. Bottom half: the absolute difference between the interpolation and the given y value at the given points. The absolute difference for the second method is orders of magnitude smaller. The error-canceled LU interpolations are very similar to the initial LU interpolation.



```

1 Coefficients found with 10 iterations of LU correction (c):
2 [ 1.76004367e+01 -1.92813615e+02  2.41090086e+02 -1.10577877e+02
3   2.69309767e+01 -4.07920689e+00  4.17991738e-01 -3.04825810e-02
4   1.63510778e-03 -6.59536551e-05  2.02950069e-06 -4.80390585e-08
5   8.77064968e-10 -1.23133950e-11  1.31634240e-13 -1.05148487e-15
6   6.07365027e-18 -2.39518661e-20  5.76783589e-23 -6.39614985e-26]

```

Vandermonde_output.txt

2.4 2d: Timing the interpolation

Using the timeit module, we execute the code from 2a, 2b and 2c 150 times each. The results are given below:

```

1 Time for 150 iterations of 2a: 1.9434962620143779
2 Time for 150 iterations of 2b: 28.369288722984493
3 Time for 150 iterations of 2c: 2.1036268759926315

```

Vandermonde_output.txt

We see that the LU decomposition method is quite quick, and the error cancelling doesn't add that much more time. Neville's algorithm is the clear loser when it comes to efficiency. When we take into account accuracy, though, it becomes apparent that Neville's algorithm does have significantly lower errors in this application. A possible cause for error with the LU decomposition method is roundoff: as we made use of the simple Crout decomposition, no smart pivoting is done to reduce divisions by very small numbers.

We also see that iteration on the LU method takes a small amount of extra time compared to the un-corrected LU method. We also see little to no improvement in error. It is probably the case that, given our implementation of the LU decomposition, we are already at the lowest possible error achievable by that method, and as such we have little to gain with the error cancelling algorithm.

All in all, we have shown the differences between our implementations of the discussed interpolation algorithms. If a low error is of utmost importance while computation time is not as much of a constraint, Neville's algorithm provides the best fit to the points. However, if you want to save time, LU decomposition offers a more efficient way to quickly find a good fit, as long as you are willing to accept a higher error.